

PySpark Code Reference

Page 1 of 2 · Module 5 – Apache Spark

SETUP

SparkSession (DataFrames / SQL)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

SparkContext (RDD API)

```
from pyspark import SparkContext

sc = SparkContext("local", "MyApp")

# or get it from the session:
sc = spark.sparkContext
```

RDDs — CREATE · TRANSFORM · ACT

Create & Transform

```
# Create
rdd = sc.parallelize([1, 2, 3, 4])
rdd = sc.textFile("data.txt")

# Narrow transformations (no shuffle)
rdd.map(lambda x: x * x)
rdd.filter(lambda x: x > 2)
rdd.flatMap(lambda x: x.split())

# Wide transformations (shuffle)
rdd.reduceByKey(lambda a, b: a + b)
rdd.groupByKey()

# Partitioning
rdd.repartition(8) # full shuffle
rdd.coalesce(4) # reduce, no shuffle
```

Actions & Caching

```
# Actions – trigger execution
rdd.collect() # all → Driver
rdd.count() # total elements
rdd.first() # first element
rdd.take(5) # first 5
rdd.saveAsTextFile("out/")

# Cache / Persist
rdd.cache() # memory only

from pyspark import StorageLevel
rdd.persist(StorageLevel.DISK_ONLY)

# cache() = persist(MEMORY_ONLY)
```

DataFrames & Spark SQL

Read · Filter · Write

```
# Read
df = spark.read.csv("f.csv",
    header=True, inferSchema=True)
df = spark.read.json("f.json")
df = spark.read.parquet("f.parquet")

# Common operations
df.filter(df.age > 25)
df.select("name", "age")
df.groupBy("dept").count()
df.orderBy("salary", ascending=False)
df.withColumn("tax", df.salary * 0.1)
df.dropDuplicates()
df.na.fill(0)

# Write
df.write.mode("overwrite").parquet("out/")
df.write.csv("out.csv")
```

Spark SQL

```
# Register as temp view, then query
df.createOrReplaceTempView("people")

spark.sql("""
SELECT name, age
FROM people
WHERE age > 30
""").show()
```

Broadcast Join (perf trick)

```
from pyspark.sql.functions import broadcast

# Send small_df to ALL executors
result = large_df.join(
    broadcast(small_df), 'id')
```

Spark Streaming · MLlib

Streaming — DStream Word Count

```
from pyspark.streaming import
StreamingContext

# 5-second batch interval
ssc = StreamingContext(sc, 5)
lines = ssc.socketTextStream("localhost",
9999)

words = lines.flatMap(lambda l: l.split())
counts = words.map(lambda w: (w,1)) \
.reduceByKey(lambda a,b: a+b)

counts.pprint() # print each batch
ssc.start()
ssc.awaitTermination()
```

MLlib — Logistic Regression

```
from pyspark.ml.classification \
import LogisticRegression

data = spark.read.format("libsvm") \
.load("sample_libsvm_data.txt")

lr = LogisticRegression()
model = lr.fit(data)
print(model.summary.accuracy)
```

Avoid Python UDFs

```
from pyspark.sql.functions import log, upper
# Use built-in functions - much faster
df.withColumn('v', log(df.value))
df.withColumn('n', upper(df.name))
```

Parquet is the best format for big data — compressed, columnar, and fastest to query.

■ Key Definitions & Concepts

Page 2 of 2 · Module 5 – Apache Spark

ARCHITECTURE

Apache Spark	Open-source distributed computing engine for fast big data processing across clusters using in-memory computation.
Driver Program	The brain of Spark — converts user code into a DAG of tasks, schedules them, sends to Executors, and collects results.
Executor	Worker process on a node — runs the actual tasks assigned by the Driver and caches data in memory for speed.
Cluster Manager	Resource allocator (YARN / Mesos / Standalone) that sits between Driver and workers, managing CPU & RAM assignments.
Worker Node	Physical/virtual machine in the cluster that hosts one or more Executor processes.
Spark Context	Entry point inside the Driver for interacting with the cluster; used to create RDDs and coordinate execution.

CORE CONCEPTS

RDD	Resilient Distributed Dataset — immutable, partitioned, fault-tolerant collection of objects; the foundation of Spark.
DataFrame	High-level abstraction with named columns (like a DB table); auto-optimized by the Catalyst engine.
DAG	Directed Acyclic Graph — Spark's logical execution plan; groups transformations into stages for optimized scheduling.
Transformation	Lazy operation that returns a new RDD/DF without executing immediately (e.g. map, filter, reduceByKey).
Action	Triggers actual execution of the recorded DAG and returns a result (e.g. collect, count, saveAsTextFile).
Lazy Evaluation	Spark only runs computations when an Action is called — allows it to optimize the full execution plan first.
Narrow Transform	No data shuffle needed; each partition maps to one output partition (map, filter). Fast and pipelined.
Wide Transform	Requires data shuffle across the network (groupByKey, reduceByKey). Expensive — minimize these.

SPARK SQL & PERFORMANCE

Spark SQL	Module for structured data — lets users run SQL queries on DataFrames using familiar SQL syntax.
Catalyst Optimizer	Spark SQL's query optimizer — auto-improves execution plans for DataFrames and SQL for maximum efficiency.

Broadcast Join	Sends a small DataFrame to every executor so joins happen locally, eliminating expensive network shuffles.
Partitioning	How data is split across nodes. <code>repartition(n)</code> = full shuffle; <code>coalesce(n)</code> = reduce partitions, no shuffle.
Predicate Pushdown	Filtering data at the source (e.g. inside Parquet files) before loading into memory — reduces disk I/O.
cache()	Stores an RDD/DF in memory so it can be reused without recomputation. Critical for iterative algorithms.

UNIFIED FRAMEWORK — MODULES

Spark Streaming	Processes live data by breaking it into micro-batches (e.g. every 5 sec), each handled as an RDD by the core engine.
DStream	Discretized Stream — the primary abstraction in Spark Streaming; represents a continuous sequence of RDDs.
MLlib	Spark's built-in scalable ML library — classification, regression, clustering, feature extraction, pipelines.
Pipeline (MLlib)	Chain of ML stages (feature engineering → model training) in one reproducible workflow object.

DATA FORMATS AT A GLANCE

CSV	Row-based text format. Easy to use but slow for big data — no compression, no column pruning.
JSON	Semi-structured row format. Good for nested data; slower than columnar formats for analytics.
Parquet	Columnar, compressed. Best for analytical queries — Spark only reads columns it needs. ✓ Recommended.
ORC	Like Parquet but optimized for Hive. Columnar + compressed — great for Hive-integrated pipelines.
Avro	Row-based binary format with schema support. Ideal for schema evolution and Kafka integration.
JDBC	Connector for SQL databases (MySQL, PostgreSQL, etc.) — Spark can read/write tables directly.